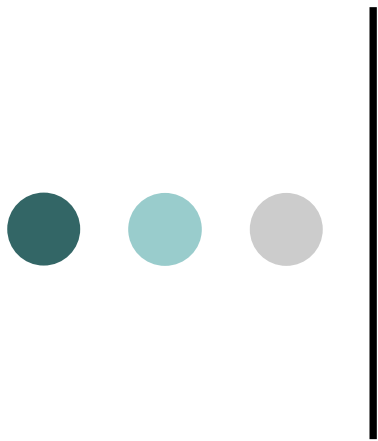




状态压缩

计算机学院 李春骐

预备知识：位运算

- 位压缩能做什么？
- 表示集合
 - 有10个物品，选出若干个物品
 - 选择1,3,7三个物品
 - 表示为 $0001000101_{(2)} = 69_{(10)}$
- 表示状态
 - 有10盏灯，一些是亮的，一些是灭的
 - 第1,3,7盏灯亮，其余灭
 - 表示为 $0001000101_{(2)} = 69_{(10)}$

预备知识：位运算

运算符

名称	符号	规则
按位与	&	全一则一，否则为零
按位或		有一则一，否则为零
按位取反	~	是零则一，是一则零
按位异或	^	不同则一，相同则零
移位	>>和<<	向右、向左移位

优先级

~

>>和<<

&

^

|

预备知识：位运算

○ 常用运算

○ 集合取并(Set union)

- $A \mid B$

○ 集合取交(Set intersection)

- $A \& B$

○ 集合相减(Set subtraction)

- $A \& \sim B$

○ 集合取反(Set negation)

- $\text{ALL_BITS} \wedge A$

预备知识：位运算

○ 常用运算

○ 置位(Set bit)

- $A \mid= 1 \ll \text{bit}$

○ 清位(Clear bit)

- $A \&= \sim(1 \ll \text{bit})$

○ 测位(Test bit)

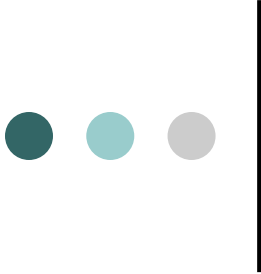
- $(A \& 1 \ll \text{bit}) \neq 0$

- $(A \gg \text{bit} \& 1) \neq 0$

预备知识：位运算

○ 其它运算

- 取最后一个非0位(Extracting every last bit)
 - $A \& -A$
 - $A \& \sim(A-1)$
- 统计非0位(Counting out the bits)
 - For (; A; A -= A & -A) ++cnt;
- 取所有子集(All the subsets)
 - $X = A$
 - While (X) $X = (X - 1) \& A$



预备知识：位运算

○ 其它运算

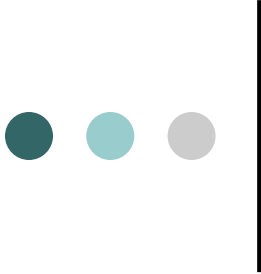
- 判断是否有相邻的1

- $(A \& A >> 1) == 0$

- 交换两整数

- $a \wedge = b, b \wedge = a, a \wedge = b$

- 还有很多很多.....

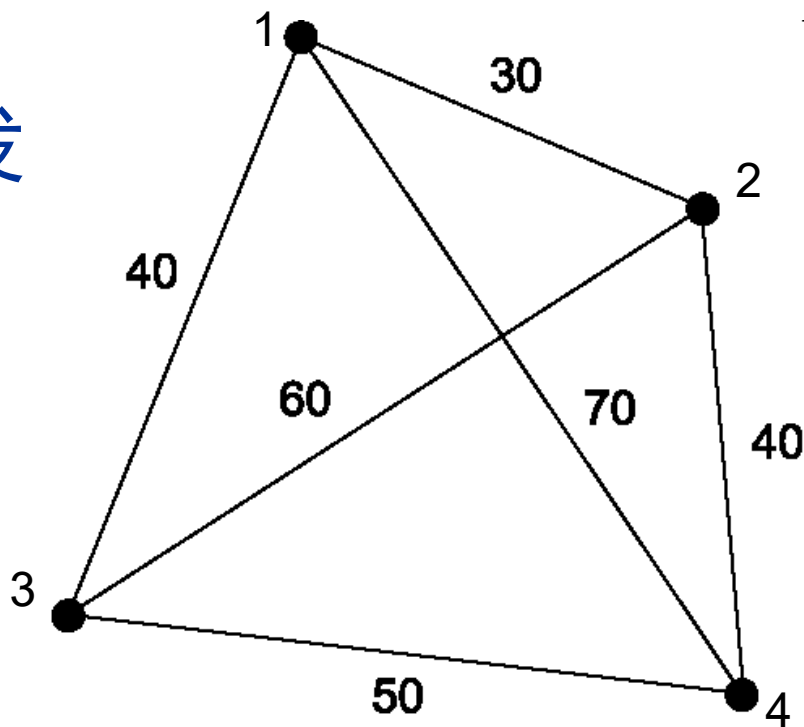


例1：旅行商问题

- 描述：旅行商要到若干个城市旅行，各城市之间的费用是已知的，为了节省费用，旅行商决定从所在城市出发，到每个城市旅行一次后返回初始城市，问他应选择什么样的路线才能使所走的总费用最短？

例1：旅行商问题

- 例子
- 从1出发



- 最优路径 1 -> 2 -> 4 -> 3

● ● ● | 例1：旅行商问题

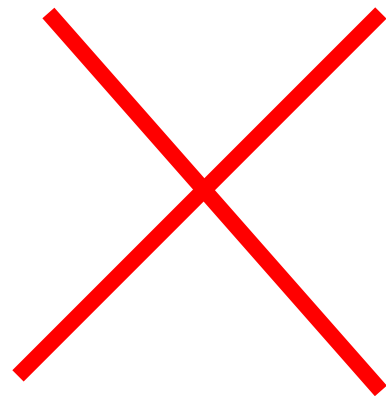
- 枚举？
- 始点固定，枚举所有路径。
- 复杂度 $O(n!)$
- 当 n 大于10



行不通！

● ● ● | 例1：旅行商问题

- 动态规划方法
- 状态定义： $dp[i]$ 到达结点 i 的最小花费。
- 正确么？
- 每个结点访问刚好一次
- 不满足无后效性



● ● ● | 例1：旅行商问题

- 状态定义： $dp[S][i]$ 表示经过点的集合为 S ，当前到达 i 号结点的最小花费。
- S 中二进制位，访问过的记1，未访问记0
- 为什么这么定义？
- 消除无后效性，记录每个结点的访问情况。

例1: 旅行商问题

○ 状态转移:

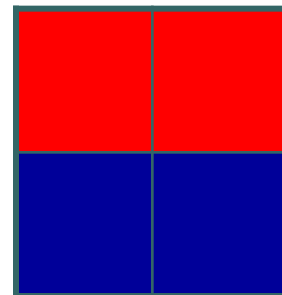
- $dp[S][i] = \min\{dp[S \setminus \{1 \leq k\}][k] + w(k, i)\}$
- 其中k满足: $S \setminus \{1 \leq k\} \neq \emptyset$

○ 复杂度($2^n * n^2$)

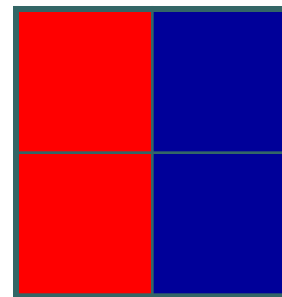
○ 当 $n > 15$ 时, 会很慢 $O(2^n * n^2)$

例2: 骨牌覆盖

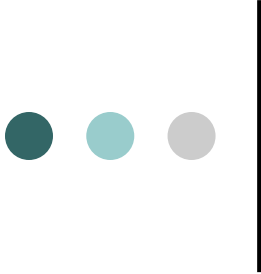
- 用 1×2 的骨牌去覆盖 $N \times M$ 的棋盘，有多少种方案。 $N, M \leq 11$ 。



- 当 $N = 1$ 时
 - M 为奇，无解， M 为偶，有唯一解



- 当 $N = 2$ 时
 - Fibonacci 数列



例2：骨牌覆盖

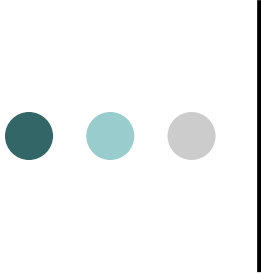
- $N > 2$ 时怎么办？
- $N \leq 11$ ，很小，可以用位压缩
- 状态表示：dp[i][S]表示到达第i行，当前覆盖状态为S的方案数。
- 对横着放的骨牌，两个位置均置为1
- 对竖着放的骨牌，上方置1，下方置0

例2: 骨牌覆盖

- 怎么转移?
- 设上一行为 $s1$, 下一行为 $s2$
- 可转移情况
- Case 1: $s1$ 某位为0, $s2$ 对应位**必须**为1
- Case 2: $s1$ 某位为1, $s2$ 对应位可为0, 可为1, **但是**需满足 $s1 \& s2$ 可被 1×2 覆盖

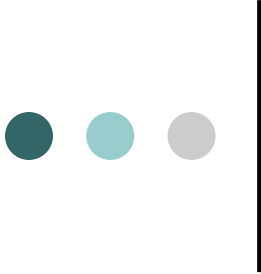
例2: 骨牌覆盖

```
LL solve(int k, int S) { // 到达第k行, 状态为S方案数
    if (dp[k][S] != -1) return dp[k][S];
    if (k == 0) {
        if (S == two(m)-1) return dp[k][S] = 1;
        else return dp[k][S] = 0;
    }
    dp[k][S] = 0;
    for (int i = 0; i < two(m); ++i)
        if ((i|S) == two(m)-1 && Cover(i&S)) //若可以转移
            dp[k][S] += solve(k-1, i);
    return dp[k][S];
}
```



例2：骨牌覆盖

```
bool Cover(int S) {  
    for (int i = 0; i < m; ++i)  
        if (S&two(i)) {  
            if (i == m-1 || !(S&two(i+1)))  
                return 0;  
            ++i;  
        }  
    return 1;  
}
```



例2：骨牌覆盖

- 另一种写法
- 先预处理出所有转移
- 如何预处理？
- DFS

例2: 骨牌覆盖

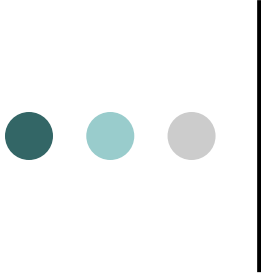
```
void dfs(int col, int s1, int s2)
{
    if (col >= m) {
        if (col == m) g[s2].push_back(s1);
        return;
    }
    dfs(col+1, (s1<<1)|1, s2<<1);
    dfs(col+1, s1<<1, (s2<<1)|1);
    dfs(col+2, (s1<<2)|3, (s2<<2)|3);
}
```

● ● ● | 例2: 骨牌覆盖

- $\text{dfs}(\text{col}+1, (s1 \ll 1) | 1, s2 \ll 1);$
- 上面是1, 下面是0, 竖放骨牌。

- $\text{dfs}(\text{col}+1, s1 \ll 1, (s2 \ll 1) | 1);$
- 上面是0, 下面是1, 分属于两个骨牌。

- $\text{dfs}(\text{col}+2, (s1 \ll 2) | 3, (s2 \ll 2) | 3);$
- 上面是11, 下面是11, 上面是横放骨牌。

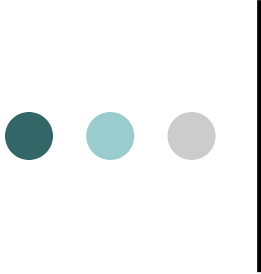


例2: 骨牌覆盖

```
for (int i = 0; i <= n; ++i) {  
    for (int j = 0; j < S; ++j) {  
        f[i+1][j] = 0;  
        for (int k = 0; k < g[j].size(); ++k)  
            f[i+1][j] += f[i][g[j][k]];  
    }  
}
```

边界: $f[0][0] = 1;$

结果: $f[n+1][(1 \ll m) - 1];$

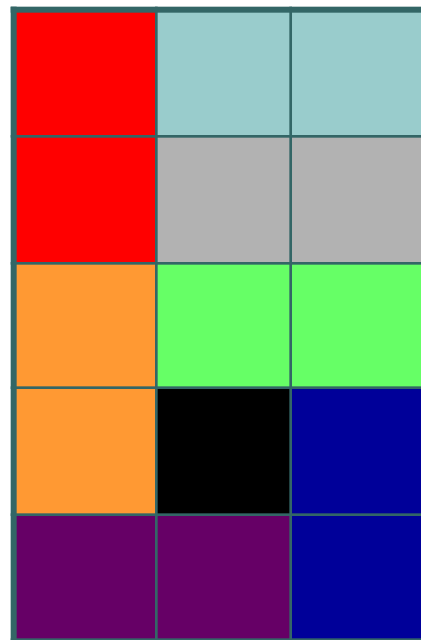
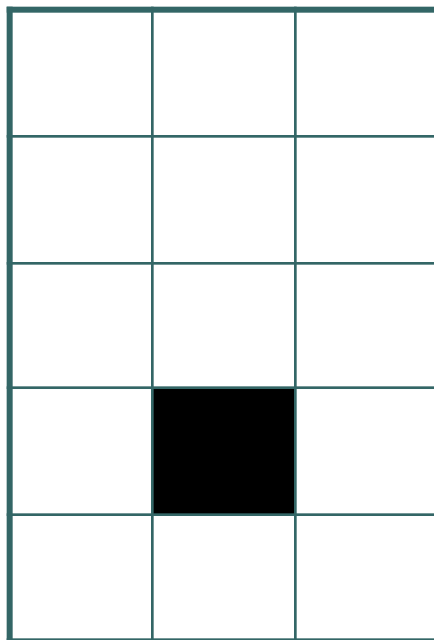


例2：骨牌覆盖

思考：上面的方法都是一行一行转移，能否一格一格的转移呢？

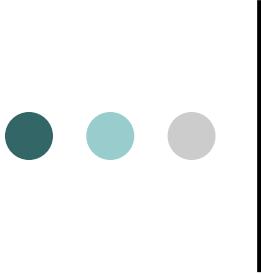
例3：禁止位置骨牌覆盖

- 用 1×2 的骨牌去覆盖 $N \times M$ 的棋盘，其中有些格子不能放。有多少种方案。 $N, M \leq 11$ 。



● ● ● | 例3：禁止位置骨牌覆盖

- 之前的方法还可不可行？
- 禁止位置如何处理？
- 相当于 1×1 的骨牌，单独一格作为一个连通分量。
- 对横着放的骨牌，两个位置均置为1
- 对竖着放的骨牌，上方置1，下方置0
- 从上一行转移下来时，禁止位置当1处理；
向下一行转移下去时，禁止位置当0处理。



例3：禁止位置骨牌覆盖

- 枚举每一种状态，搜出所有的可行状态转移。

例3：禁止位置骨牌覆盖

```
void dfs(int col, int s1, int s2, int i)
{
    if (col == m) {
        dp[i+1][s2^mask[i]] += dp[i][s1];
        return;
    }
    if (col < m-1 && (s1&two(col)) && (s1&two(col+1)))
        dfs(col+2, s1, (s2<<2)|3, i);
    if ((s1&two(col)) && !(mask[i]&two(col)))
        dfs(col+1, s1, s2<<1, i);
    if (!(s1&two(col)))
        dfs(col+1, s1, (s2<<1)|1, i);
}
```

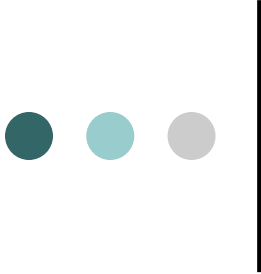
例3：禁止位置骨牌覆盖

- 上面一行11，下面一行也是11
 - `if (col < m-1 && (s1&two(col)) && (s1&two(col+1)))`
 - `dfs(col+2, s1, (s2<<2)|3, i);`
- 上面一行1，下面一行0
 - `if ((s1&two(col)) && !(mask[i]&two(col)))`
 - `dfs(col+1, s1, s2<<1, i);`
- 上面一行0，下面一行1
 - `if (!(s1&two(col)))`
 - `dfs(col+1, s1, (s2<<1)|1, i);`

● ● ● | 例3：禁止位置骨牌覆盖

○ 递推过程

```
for (int i = 0; i <= n; ++i)
    for (int s = 0; s < 1<<m; ++s)
        if (dp[i][s] > 0)
            dfs(0, s, 0, i);
```



例3：禁止位置骨牌覆盖

- 思考1：先预处理出所有转移可不可以？
- 思考2：一格一格的推怎么做呢？

例3：禁止位置骨牌覆盖

```
dfs(0, 0, 0); r1 = 0, r0 = 1; f[0][0] = 1;
for (int i = 0; i <= n; ++i) {
    r0 ^= 1, r1 ^= 1;
    memset(f[r1], 0, sizeof(f[r1]));
    for (int j = S-1; j >= 0; --j)
        if ((j & mask[i]) == mask[i]) {
            for (int k = 0; k < g[j].size(); ++k)
                f[r1][j] += f[r0][g[j][k]];
            if (mask[i]) {
                f[r1][j ^ mask[i]] += f[r1][j];
                f[r1][j] = 0;
            }
        }
    }
}
```